

3. Offline Optimization Algorithm

This section presents the offline optimization procedure in a journal-style narrative consistent with the current implementation. Section 3.2 (ETPRC initial construction) is intentionally omitted from this chapter because it is documented separately; here, ETPRC is treated as a provided initializer and as a rebuild primitive.

3.1. Offline Objective for Algorithmic Optimization

The full optimization model has been formally defined in the earlier problem formulation section. In the implemented offline optimizer, the search objective is operationalized as

$$\min Z = Z_{\text{fixed}} + Z_{\text{truck}} + Z_{\text{drone}} + Z_{\text{fail}}^{\text{exp}} + \beta Z_{\text{tw}} + Z_{\text{unserved}}, \quad (1)$$

where

$$Z_{\text{fixed}} = c_{\text{pair}} \cdot |\mathcal{P}_{\text{used}}|, \quad (2)$$

$$Z_{\text{truck}} = \sum_{(i,j) \in \mathcal{A}_T} c_T d_T(i, j), \quad (3)$$

$$Z_{\text{drone}} = \sum_{s \in \mathcal{S}} c_E E_s, \quad (4)$$

$$Z_{\text{fail}}^{\text{exp}} = \sum_{i \in \mathcal{N}_{\text{served}}} (1 - p_i(t_i^*)) Z_{\text{fail}}(i), \quad (5)$$

$$Z_{\text{fail}}(i) = 2 d_T(0, i) c_T, \quad (6)$$

$$Z_{\text{tw}} = \sum_{i \in \mathcal{N}_{\text{served}}} \max(0, a_i - l_i), \quad (7)$$

$$Z_{\text{unserved}} = \sum_{i \in \mathcal{N}_{\text{unserved}}} 2 Z_{\text{fail}}(i). \quad (8)$$

The time-window penalty coefficient is dynamically updated along ACO outer iterations:

$$\beta \leftarrow \min(\gamma_\beta \beta, \beta_{\text{max}}). \quad (9)$$

This schedule encourages exploration in earlier phases and progressively strengthens temporal discipline in later phases.

Recommended figure placement.. Figure 3-1: Objective decomposition and information flow from route/sortie/timing states to each cost term.

3.2. ACO Outer Construction and Pheromone Learning

The global search layer follows an ACO outer loop. In each outer iteration, multiple ants construct candidate solutions, each candidate is locally refined by ALNS, and the best ant outcome updates the global incumbent and pheromone matrix.

3.2.1. Ant Construction Policy

For each customer group, route extension follows

$$w_{ij} = \tau_{ij}^{\alpha_{aco}} \eta_{ij}^{\beta_{aco}}, \quad (10)$$

where τ_{ij} is pheromone and η_{ij} is a weighted heuristic combining distance slack, time-window slack, and capacity slack. Candidate selection uses roulette-wheel sampling. If no feasible option exists in the near-candidate list, the candidate pool is expanded; if assignment quality degrades, the implementation falls back to a robust ETPRC-based construction path.

3.2.2. Pheromone Update Rule

After each outer iteration, the algorithm applies:

1. global evaporation on all arcs,
2. deposition from iteration-best solution,
3. weighted deposition from global-best solution,
4. clipping to $[\tau_{\min}, \tau_{\max}]$.

A compact expression is

$$\tau \leftarrow \text{clip}((1 - \rho)\tau + \Delta\tau_{\text{iter}} + \omega_g \Delta\tau_{\text{global}}, \tau_{\min}, \tau_{\max}). \quad (11)$$

Recommended figure placement.. Figure 3-2: ACO workflow (construction $\rightarrow$ ALNS refinement $\rightarrow$ selection $\rightarrow$ pheromone update $\rightarrow$ beta update).

Figure 3-3: Pheromone heatmap evolution between two consecutive outer iterations.

3.3. ALNS Local Improvement Mechanism

Each ant solution enters an ALNS inner loop for local refinement. Per ALNS iteration:

1. one destroy and one repair operator are sampled by roulette-wheel from adaptive weights,
2. removal size q is sampled in $[q_{\min}, q_{\max}]$,
3. destroy and repair are executed,
4. fallback rebuild is triggered if unserved customers or structural damage appears,
5. periodic cross-group perturbation is optionally applied,
6. acceptance is decided by simulated annealing,
7. selected operator weights and temperature are updated.

The implementation distinguishes between the *current accepted state* (exploratory) and the *best local return state*, which must pass hard-feasibility checks.

Recommended figure placement.. Figure 3-4: ALNS state-transition graph with acceptance/rejection branches.

3.4. Neighborhood Operator Design (D1–D4, R1–R3, Cross-group)

3.4.1. Destroy Operators

- **D1 Random Removal:** removes q randomly sampled customers.
- **D2 Worst Removal:** ranks customers by objective relief under hypothetical single-customer removal and removes top contributors.
- **D3 Related Removal:** starts from a random seed and removes geographically related customers.
- **D4 Low-Probability Removal:** computes $p_i(t_i^*)$ and removes the lowest-probability customers.

3.4.2. Repair Operators

- **R1 Greedy Insertion:** selects minimum-objective feasible insertion at each step.
- **R2 Regret Insertion:** prioritizes

$$\text{regret}(i) = Z_{\text{2nd}}(i) - Z_{\text{best}}(i). \quad (12)$$

- **R3 Timeslot-aware Insertion:** prioritizes larger $p_i(t_i^*)$, with objective as tie-breaker.

3.4.3. Cross-group Operators

- **Cross-pair Swap:** swaps small customer subsets between two pairs; accepted only if strict improvement and feasibility hold.
- **Cross-pair Transfer:** transfers customers between pairs; accepted only if strict improvement and feasibility hold.

Recommended figure placement.. Figure 3-5: Before/after neighborhood transformations for D/R/Cross operators.

Figure 3-6: Two-pair swap/transfer case study.

3.5. Acceptance Criterion and Adaptive Weight Update

3.5.1. Simulated Annealing Acceptance

Let

$$\Delta = Z_{\text{new}} - Z_{\text{cur}}. \quad (13)$$

If $\Delta < 0$, the candidate is accepted deterministically; otherwise, acceptance is probabilistic:

$$P_{\text{accept}} = \exp\left(-\frac{\Delta}{T}\right). \quad (14)$$

Temperature cooling is:

$$T \leftarrow \max(T \cdot \alpha_{\text{cool}}, \epsilon). \quad (15)$$

3.5.2. Reward Scheme

Selected operators receive:

- σ_1 : new global best,
- σ_2 : local improvement,
- σ_3 : accepted worsening move,
- 0: rejected or invalid candidate.

3.5.3. Adaptive Weight Update

For selected destroy/repair operators:

$$w \leftarrow (1 - \lambda_w)w + \lambda_w \cdot \text{reward}. \quad (16)$$

This provides online credit assignment and adaptive bias balancing.

Recommended table placement.. Table 3-1: Reward class, acceptance condition, and weight-update impact.

3.6. Feasibility Safeguard Mechanism

In the updated implementation, feasibility is no longer represented as isolated standalone repair operators. Instead, a layered safeguard pipeline is used:

1. structural damage detection for sortie anchors/order consistency,
2. fallback rebuild of affected pairs via ETPRC primitives,
3. structural/physical pre-check before objective evaluation,
4. hard-feasible gating for best-local promotion.

Hence, feasibility assurance is centralized into a robust and reusable mechanism rather than fragmented operator-specific patches.

Recommended figure placement.. Figure 3-7: Feasibility pipeline (Detect $\rightarrow$ Rebuild $\rightarrow$ Pre-check $\rightarrow$ Best-feasible gate).

Figure 3-8: Damaged sortie structure vs repaired structure.

3.7. Stopping Rules, Complexity, and Parallelization

3.7.1. Stopping Rules

The offline ACO+ALNS solver terminates when either condition is met:

1. maximum ACO outer iterations reached,
2. no-improvement counter reaches $N_{\text{no-improve}}$.

Within each ant, ALNS executes a fixed number N_{alns} of local iterations.

Algorithm 1 Overall Offline Optimization (ACO + ALNS)

Require: instance, parameters, all-home policy, random seed

Ensure: best feasible offline solution S_{best}

```
1:  $S_0 \leftarrow \text{ETPRC\_InitialSolution}(\text{instance}, \text{parameters})$ 
2:  $Z_0 \leftarrow \text{Objective}(S_0, \beta_{\text{init}})$ 
3: Initialize pheromone  $\tau$ ,  $\beta \leftarrow \beta_{\text{init}}$ 
4:  $S_{\text{best}} \leftarrow S_0$ ,  $Z_{\text{best}} \leftarrow Z_0$ ,  $\text{noImprove} \leftarrow 0$ 
5: for  $\text{iterACO} = 1$  to  $I_{\text{aco}, \text{max}}$  do
6:   Build ant tasks with shared  $\tau$  and current  $\beta$ 
7:   Run ant tasks in parallel (or serial fallback)
8:   Select iteration-best  $(S_{\text{iter}}, Z_{\text{iter}})$ 
9:   if  $Z_{\text{iter}} < Z_{\text{best}}$  then
10:     $S_{\text{best}} \leftarrow S_{\text{iter}}$ ,  $Z_{\text{best}} \leftarrow Z_{\text{iter}}$ ,  $\text{noImprove} \leftarrow 0$ 
11:   else
12:     $\text{noImprove} \leftarrow \text{noImprove} + 1$ 
13:   end if
14:    $\tau \leftarrow \text{PheromoneUpdate}(\tau, S_{\text{iter}}, Z_{\text{iter}}, S_{\text{best}}, Z_{\text{best}})$ 
15:    $\beta \leftarrow \min(\beta \cdot \gamma_{\beta}, \beta_{\text{max}})$ 
16:   if  $\text{noImprove} \geq \text{noImprove}_{\text{max}}$  then
17:     break
18:   end if
19: end for
20: return  $S_{\text{best}}$ 
```

3.7.2. Complexity Characterization

Let N be customer count, A ant count, I ACO outer iterations, and L ALNS local iterations. A practical decomposition is

$$\mathcal{T}_{\text{offline}} \approx I \cdot A \cdot (\mathcal{T}_{\text{construct}}(N) + \mathcal{T}_{\text{ALNS}}(L, N)) + I \cdot \mathcal{T}_{\text{pheromone}}. \quad (17)$$

In practice, candidate generation, feasibility checks, objective recomputation, and fallback-trigger frequency dominate runtime.

3.7.3. Parallelization Strategy

Ant-level multiprocessing is used:

- parallel unit: one ant task (construct + ALNS refine),
- worker count: up to $\min(A, \text{CPU} - 1)$,
- synchronization barrier: end of each ACO iteration,
- fault handling: serial fallback when parallel execution fails.

Recommended figure placement.. Figure 3-9: Parallel ant execution timeline with per-iteration synchronization.

Algorithm 2 ACO Constructive Procedure

Require: grouped customers, τ , heuristic weights

Ensure: complete candidate solution S

```
1: for each pair-group  $G$  do
2:   route  $\leftarrow$  [depot], remaining  $\leftarrow G$ 
3:   while remaining  $\neq \emptyset$  do
4:     Build near-candidate list  $C$ 
5:     Compute transition weights  $w_{ij}$  for  $j \in C$ 
6:     Roulette-select next customer  $j^*$ 
7:     Append  $j^*$ ; update time/load states
8:   end while
9:   Close route and extract sorties
10: end for
11: if unserved customers exist then
12:   fallback to robust ETPRC construction
13: end if
14: return  $S$ 
```

Readability and visual communication note.. To preserve the explanatory strength previously achieved by dedicated feasibility-repair figures, the revised chapter can use: (i) one unified feasibility pipeline figure; (ii) one before/after fallback-rebuild snapshot; (iii) one constraints-coverage matrix (pre-check vs hard feasibility vs soft-penalty objective); and (iv) one operator-transformation panel for D/R/Cross exemplars.

Algorithm 3 ALNS Local Search Procedure

Require: $S_{\text{cur}}, Z_{\text{global}}, \beta$ **Ensure:** locally best feasible solution S_{best}

```
1: Initialize destroy/repair weights and temperature  $T$ 
2:  $S_{\text{best}} \leftarrow S_{\text{cur}}$ 
3: for  $k = 1$  to  $N_{\text{alns}}$  do
4:   Select destroy  $D$  and repair  $R$  by roulette
5:   Sample  $q \in [q_{\min}, q_{\max}]$ 
6:    $S_{\text{part}} \leftarrow D(S_{\text{cur}}, q)$ 
7:    $S_{\text{new}} \leftarrow R(S_{\text{part}})$ 
8:   if unserved customers or structural damage then
9:      $S_{\text{new}} \leftarrow \text{FallbackRebuild}(S_{\text{new}})$ 
10:  end if
11:  if periodic cross condition holds then
12:     $S_{\text{new}} \leftarrow \text{CrossGroupPerturbation}(S_{\text{new}})$ 
13:  end if
14:  if pre-check fails then
15:    reject and set reward = 0
16:  else
17:     $Z_{\text{new}} \leftarrow \text{Objective}(S_{\text{new}}, \beta)$ 
18:    accept  $\leftarrow \text{SA\_Accept}(Z_{\text{cur}}, Z_{\text{new}}, T)$ 
19:    reward  $\leftarrow \text{RewardClass}(\text{accept}, Z_{\text{new}}, Z_{\text{global}})$ 
20:  end if
21:  if accept then
22:     $S_{\text{cur}} \leftarrow S_{\text{new}}$ 
23:  end if
24:  if accept and hard-feasible( $S_{\text{cur}}$ ) and better than  $S_{\text{best}}$  then
25:     $S_{\text{best}} \leftarrow S_{\text{cur}}$ 
26:  end if
27:  Update selected operator weights
28:   $T \leftarrow T \cdot \text{cooling\_rate}$ 
29: end for
30: return  $S_{\text{best}}$ 
```

Algorithm 4 SA Acceptance and Adaptive Weight Update

Require: $Z_{\text{cur}}, Z_{\text{new}}, Z_{\text{global}}, T, w_D, w_R$ **Ensure:** accept flag and updated weights

```
1:  $\Delta \leftarrow Z_{\text{new}} - Z_{\text{cur}}$ 
2: if  $\Delta < 0$  then
3:   accept  $\leftarrow$  true
4:   reward  $\leftarrow \sigma_1$  if  $Z_{\text{new}} < Z_{\text{global}}$ , else  $\sigma_2$ 
5: else
6:    $p \leftarrow \exp(-\Delta/T)$ 
7:   accept  $\leftarrow (\text{rand}() < p)$ 
8:   reward  $\leftarrow \sigma_3$  if accept else 0
9: end if
10:  $w_D \leftarrow (1 - \lambda_w)w_D + \lambda_w \cdot \text{reward}$ 
11:  $w_R \leftarrow (1 - \lambda_w)w_R + \lambda_w \cdot \text{reward}$ 
12: return accept,  $w_D, w_R$ 
```

Algorithm 5 Feasibility Safeguard and Fallback Rebuild

Require: candidate solution S **Ensure:** repaired candidate S'

```
1: Detect damaged pairs and unserved set
2: if no damage and no unserved then
3:   return  $S$ 
4: end if
5: Assign unserved customers to suitable pairs
6: for each affected pair do
7:   Rebuild truck route and sorties via ETPRC primitives
8:   if rebuilt structure remains invalid then
9:     clear sorties and move affected customers to truck route
10:  end if
11: end for
12: Recompute global unserved set from assignments
13: return  $S'$ 
```
